Welcome to this course on Phaser 4, an open-source HTML5 game framework that makes building 2D games for the web both fun and accessible. Whether you want to create your own platformers, design interactive prototypes, or build games to showcase in your portfolio, Phaser 4 provides the tools you need to bring your ideas to life.

What Makes Phaser 4 Special?

Phaser 4 is a powerful choice for web game development for several key reasons:

- **Cross-Platform Compatibility:** Build your game once and it will run in any modern web browser, allowing you to play anywhere.
- **Free and Open Source:** Phaser has a vibrant community and is completely free to use for any project.
- **Ideal for 2D Games:** From platformers and arcade classics to puzzle games, Phaser 4 is perfectly suited for building classic-style 2D games.
- **Modern JavaScript:** Built on the latest JavaScript standards, the framework is efficient and enjoyable to work with.
- **Feature-Rich:** We will explore a wide range of built-in features, including physics, animations, input management, scene management, tilemaps, and object pooling to create polished gameplay.

What Will You Be Building?

Throughout this course, you will create your very own Mario-style platformer game from scratch. You will guide your character through levels, help them avoid hazards, and allow them to interact with objects like barrels and collectibles. Along the way, you will learn to implement core platformer mechanics.

- Implementing player walking and jumping.
- Detecting collisions and overlaps.
- Using tilemaps to build levels.
- Creating dynamic and static objects.
- Animating sprites and handling player input.
- Generating objects dynamically and recycling them efficiently.

By the end of the course, you will have a complete, working platformer that you can expand with your own levels, hazards, or power-ups, or use as a foundation for your original game projects.

What Will You Learn in This Course?

We will walk through every step of building a platformer, focusing on key concepts and practical skills. You will learn how to:

- Master Phaser’s Arcade Physics system to handle gravity, movement, and collisions.
- Implement collision and overlap checks for players, platforms, and interactive objects.
- Build classic platformer mechanics like walking, jumping, and blocking against world bounds.
- Create timed spawners to generate barrels and other obstacles at set intervals.
- Use object pooling to recycle objects and optimize game performance.
- Load level data from JSON files, allowing you to design and expand your game worlds with ease.

By the end, you will not only know how to build these features but also understand why they work and how to adapt them for your own projects.

Prerequisites

© 2025 Zenva Pty Ltd. All rights reserved.

Before we begin, here is what you will need to follow along successfully:

- Basic to intermediate HTML, CSS, and JavaScript skills.
- Basic familiarity with Phaser 4 is helpful but not strictly required.
- A code editor, such as Visual Studio Code.
- A local web server to run your game files. We will cover how to set this up using the Live Server extension for VS Code early in the course. If you have a preferred IDE or web server setup, that will also work.

About Zenva

Zenva is an online learning academy with over a million students, featuring a wide range of courses for beginners and experienced developers alike. Our courses are versatile, allowing you to learn in the way that suits you best. You can watch video tutorials, read lesson summaries, and follow along with the included project files. We are excited to have you on this Phaser 4 journey!

© 2025 Zenva Pty Ltd. All rights reserved.

Before we begin coding, let’s take a moment to review the game we will be building together in this course: a Mario-style platformer built with Phaser 4. This overview will cover the overall game concept, the core mechanics, and how we will structure the scenes for our game. By the end, you will have a clear picture of what we are building and how all the pieces fit together.

Game Concept

Our project is inspired by classic arcade platformers like *Super Mario Bros.* and *Donkey Kong*. The player will control a character who can walk, jump, and navigate platforms while avoiding obstacles. Along the way, we will introduce hazards like rolling barrels and design a level layout that challenges the player’s movement and timing. This creates a simple but powerful framework that you can later expand into your own unique platformer adventure.

Core Mechanics

Here are the core mechanics we will be building step-by-step. These mechanics will provide a complete foundation for fun and replayable platformer gameplay.

- **Walking and Jumping:** We will use Phaser’s Arcade Physics engine to move our player character.
- **Platforms and Obstacles:** This will cover the static and moving objects the player interacts with.
- **Collisions and Overlaps:** We will learn to detect when the player lands on a platform, collides with a hazard, or triggers other events.
- **Barrel Spawner:** We will use timed events to release barrels, creating a challenge for the player.
- **Object Pooling:** To optimize performance, we will implement a system to recycle barrels instead of constantly creating and destroying them.

Game Structure and Scenes

To keep our project organized, we will split the game into multiple scenes. Each scene will have a specific responsibility.

- **Loading Scene:** This scene will be responsible for pre-loading all the assets (images, sounds, etc.) we need for our game.
- **Game Scene:** This is where the core gameplay will take place and all the action will happen.

This is the plan for our project. Now that you have an understanding of what we are building and how it is structured, we are ready to begin. Let’s jump in and start creating your platformer.

© 2025 Zenva Pty Ltd. All rights reserved.

In any development journey, the first step is always to establish a solid project environment. A proper setup is a crucial foundation that will prevent potential issues and streamline your workflow. This article will guide you through the process of preparing your environment for creating interactive 2D games with Phaser.

Prerequisites

To begin, you will need to have a few tools and files ready. Please ensure you have the following:

- **A Code Editor:** We recommend using Visual Studio Code (VS Code), but you are welcome to use any code editor you prefer.
- **A Local Web Server:** We will be using the Live Server extension for VS Code. If you have an alternative local web server setup, you may use that instead.
- **Project Files:** Please download the starter project files and unzip them to an easily accessible location on your computer.

Note that this guide will specifically demonstrate the setup using VS Code and the Live Server extension.

Why Use a Local Web Server?

When creating web-based games with a framework like Phaser, simply opening the index.html file directly in a web browser can lead to limitations, especially as your game logic and asset management become more complex. A local web server provides a more robust and reliable development environment for several key reasons:

- **Serving Files Correctly:** A server ensures that your HTML, CSS, and JavaScript files are served properly, allowing all assets and dependencies to be loaded as intended. This is essential for Phaser to function correctly.
- **Handling Relative Paths:** Web servers correctly interpret relative file paths within your project. This prevents common issues where images, scripts, or other resources fail to load when opening the HTML file directly from your file system.
- **Enabling Live Reloading:** Tools like Live Server can automatically refresh your browser whenever you save changes to your code. This instant feedback loop significantly accelerates the development process by immediately showing you the results of your modifications.

Installing the Live Server Extension

To streamline our development process, we will use the Live Server extension for VS Code. This tool automatically refreshes your web browser each time you save your code, making development smoother and faster. Follow these steps to install it:

1. Open VS Code and navigate to the Extensions marketplace (the icon with four squares in the side panel).
2. In the search bar, type “Live Server”.
3. Locate the extension by Ritwick Dey and click the “Install” button.
4. Once the installation is complete, you should see a “Go Live” button appear in the status bar at the bottom-right corner of your VS Code window.

Exploring the Project Files

Now that your tools are ready, let’s examine the structure of the starter project files. After unzipping the folder, you will find the following components:

- **Phaser Library (phaser.min.js):** This file is the core of the Phaser game development

© 2025 Zenva Pty Ltd. All rights reserved.

framework. It is a collection of code that handles complex tasks for us, from rendering graphics to managing game logic.
- **Webpage (index.html):** This is the entry point for our game. It is a standard HTML document that links to our CSS and JavaScript files. The game itself will be rendered inside a canvas element within the body of this document.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <style>
    body {
      margin: 0;
    }
    canvas {
      width: 100%;
    }
  </style>
</head>
<body>
  <script src="src/lib/phaser.min.js"></script>
  <script src="src/main.js" type="module"></script>
</body>
</html>
```

- **Assets Folder (assets/images/):** This folder contains all the image files we will use in our game.
- **Main JavaScript File (main.js):** This file is where we create our Phaser game configuration and initialize a new Phaser game instance. It links to our game's different scenes.

```
import { GameScene } from './scenes/game-scene.js';
import { LoadingScene } from './scenes/loading-scene.js';

// our game's configuration
const config = {
  type: Phaser.AUTO,
  scene: [LoadingScene, GameScene],
  title: 'Monster Kong',
  pixelArt: false,
  scale: {
    width: 360,
    height: 640,
    autoCenter: Phaser.Scale.CENTER_BOTH,
    mode: Phaser.Scale.FIT,
  },
};

// create the game, and pass it the configuration
const game = new Phaser.Game(config);
```

- **Scenes Folder (src/scenes/):** This folder contains the individual scenes for our game.
  - **LoadingScene.js:** This scene is responsible for preloading all necessary game assets, such as images and sprite sheets. Once all assets are loaded, it starts the

© 2025 Zenva Pty Ltd. All rights reserved.

main game scene.

```
export class LoadingScene extends Phaser.Scene {
  constructor() {
    super({ key: 'LoadingScene' });
  }

  // load asset files for our game
  preload() {
    // load images
    this.load.image('ground', 'assets/images/ground.png');
    this.load.image('platform', 'assets/images/platform.png');
    this.load.image('block', 'assets/images/block.png');
    this.load.image('goal', 'assets/images/gorilla3.png');
    this.load.image('barrel', 'assets/images/barrel.png');

    // load spritesheets
    this.load.spritesheet('player', 'assets/images/player_spritesheet.png', {
      frameWidth: 28,
      frameHeight: 30,
      margin: 1,
      spacing: 1
    });

    this.load.spritesheet('fire', 'assets/images/fire_spritesheet.png', {
      frameWidth: 20,
      frameHeight: 21,
      margin: 1,
      spacing: 1
    });
  }

  // executed once, after assets were loaded
  create() {
    this.scene.start("GameScene")
  }
}
```

GameScene.js: This is where the main gameplay will occur. Currently, it is set up to simply log a message to the developer console.

```
export class GameScene extends Phaser.Scene {
  constructor() {
    super({ key: 'GameScene' });
  }

  // some parameters for our scene
  init() {}

  // executed once, after assets were loaded
  create() {
    console.log('GameScene create method called');
  }
}
```

© 2025 Zenva Pty Ltd. All rights reserved.

Running Your Project

With the environment set up and an understanding of the project structure, let's launch the game in a browser using Live Server.

1. In VS Code, with your project folder open, click the "Go Live" button in the bottom-right status bar.
2. VS Code will automatically open your default web browser and serve your project from a local server address, which will look something like http://127.0.0.1:5500.

If everything is configured correctly, you should see a black rectangle centered on the web page. This is the canvas element where our Phaser game will be rendered. You can confirm this by opening your browser's developer tools and inspecting the page elements; you will find a canvas tag inside the console.

Lesson Recap

Congratulations, you now have a proper development environment ready for building an amazing game with Phaser! In this lesson, we have:

- Installed the Live Server extension in VS Code.
- Downloaded and unzipped the starter project files.
- Toured the project structure to understand the role of each file.
- Used Live Server to run the initial game project in a browser.

In the next lesson, we will begin the exciting process of creating game objects for our game.

© 2025 Zenva Pty Ltd. All rights reserved.

In this lesson, we will explore one of the most important components of Phaser 4: the Arcade Physics system. This system is responsible for the movement, gravity, and collision of our game objects, providing the core mechanics for a platformer game. By default, a new Phaser 4 game does not have physics enabled, so we must first configure it.

Phaser's Physics Engines

Phaser comes with two built-in physics systems. It is important to understand the purpose of each to choose the right one for your project.

- **Arcade Physics:** As its name suggests, this system is designed for arcade or retro-style games. It is a lightweight physics engine that primarily handles rectangular and circular physics shapes. While not intended for complex physics simulations, it is very fast, easy to use, and perfect for platformers, top-down shooters, and puzzle games.
- **Matter.js:** This is an open-source, third-party physics library bundled with Phaser. Matter.js provides a more advanced, full-body physics system. If your game requires complex physics shapes, constraints, joints, or other advanced behaviors, Matter.js is the appropriate choice.

For this course, we will be focusing on the Arcade Physics system.

Enabling and Configuring Arcade Physics

To use a physics system, we must first enable it in our game's configuration object. This is done in the main.js file. We will add a new property called physics to our configuration object.

Inside the physics object, we specify the default system we want to use, which in our case is 'arcade'. We can then provide specific configuration for that system by adding a property with the same name, arcade.

```
const config = {
  type: Phaser.AUTO,
  scene: [LoadingScene, GameScene],
  title: 'Monster Kong',
  pixelArt: false,
  scale: {
    width: 360,
    height: 640,
    autoCenter: Phaser.Scale.CENTER_BOTH,
    mode: Phaser.Scale.FIT,
  },
  physics: {
    default: 'arcade',
    arcade: {
      gravity: {
        x: 0,
        y: 100,
      },
      debug: true,
    }
  }
};
```

In the configuration above, we have set two global properties for the Arcade Physics system:

© 2025 Zenva Pty Ltd. All rights reserved.

- **gravity:** This object defines the default gravitational force applied to all physics-enabled objects in the game. We have set a vertical gravity of 100.
- **debug:** Setting this to true is incredibly helpful during development. It visually renders the physics bodies and velocity vectors of your game objects.

Adding Physics to Game Objects

Now that the physics system is enabled, let's see how it affects our game objects. Navigate to the create method in your GameScene class (in game-scene.js) and add a sprite for the ground.

```
create() {
  const ground = this.add.sprite(180, 400, 'ground');
}
```

If you run the game, you will notice that even with gravity set, the ground sprite remains stationary. This is because, by default, game objects are not part of the physics simulation. You must explicitly enable physics for each object that needs to be affected by it.

To enable physics on an existing game object, we use the physics plugin's factory method, existing.

```
create() {
  const ground = this.add.sprite(180, 400, 'ground');
  this.physics.add.existing(ground);
}
```

With this change, the ground sprite will immediately start falling downwards due to the global gravity we configured.

Understanding Gravity and Debug Mode

The gravity property in the physics configuration has a direct impact on all dynamic physics bodies.

- A positive y value will pull objects down. A higher value results in a stronger gravitational pull.
- A negative y value will cause objects to float upwards.
- A positive x value will pull objects to the right, while a negative x value will pull them to the left.

Because we set debug: true, you will see a purple box around the ground sprite. This box represents the object's **physics body**, which is the shape used for collision detection. You may also see a green line, which indicates the object's velocity and direction of movement.

The Physics Body Property

When you add a game object to the physics system, Phaser attaches a new body property to it. This property is an object that contains a wealth of information and controls for the object's physics, such as its velocity, collision settings, and whether gravity is enabled for it. If physics is not enabled for an object, its body property will be null.

You can inspect this by logging the game object to the console.

© 2025 Zenva Pty Ltd. All rights reserved.

```
create() {
  const ground = this.add.sprite(180, 400, 'ground');
  this.physics.add.existing(ground);
  console.log(ground);
}
```

A More Direct Approach: The Arcade Sprite

Instead of creating a sprite and then separately enabling physics on it, you can accomplish both in a single step. The physics plugin has its own factory for creating sprites that are physics-enabled by default. These are known as Arcade Sprites.

```
create() {
  const ground2 = this.physics.add.sprite(180, 400, 'ground');
  console.log(ground2);
}
```

This line of code creates a sprite and immediately gives it a physics body. An Arcade Sprite has all the same properties and methods as a regular sprite, with the addition of the physics body. Using this.physics.add.sprite() is generally the preferred method if you know an object will require physics from the moment it is created.

We have now covered the basics of setting up and using the Arcade Physics system. In the next lesson, we will differentiate between static and dynamic bodies to create solid platforms that other objects can interact with.

© 2025 Zenva Pty Ltd. All rights reserved.

Welcome back! In the previous lesson, we successfully set up our arcade physics system and added our first physics-enabled sprites. However, you may have noticed that all objects are currently affected by gravity and fall off the screen. In this lesson, we will address this by learning how to create static and dynamic bodies and how to use collisions to build a stable game world.

Controlling Gravity on Dynamic Bodies

At present, our ground game objects are created as sprites with dynamic physics bodies. This means they are fully simulated by the physics engine and are affected by forces like gravity. Let's start by modifying one of our ground objects to prevent it from falling. First, ensure you have two ground objects in your scene for demonstration purposes.

```
// In your create() method
const ground = this.physics.add.sprite(180, 400, 'ground');
const ground2 = this.physics.add.sprite(180, 200, 'ground');
```

To prevent the first ground object from being affected by gravity, we can access its physics body and disable gravity specifically for that object. This is done using the setAllowGravity method.

```
ground.body.setAllowGravity(false);
```

By passing false to this method, we instruct Phaser's physics engine to ignore this specific game object when applying global gravity. As a result, the first ground object will remain fixed at its initial position, while the second one will still fall. However, you will notice that the falling object simply passes through the stationary one.

Adding Collisions

By default, physics-enabled objects in Phaser do not collide with each other. We must explicitly define which objects should interact. To make our two ground objects collide, we need to add a physics collider.

You can add a collider through the physics plugin's factory. The collider method requires at least two arguments: the two objects or groups of objects that you want to enable collisions for.

```
this.physics.add.collider(ground, ground2);
```

With this line added, if you run the game, you will see the second ground object fall and land on top of the first one. They no longer pass through each other. However, a new issue arises: the impact from the falling object transfers its momentum to the stationary one, causing both to move. This happens because even though we disabled gravity, the object can still be affected by other physics forces, such as collisions.

Making Bodies Immovable

To create a truly solid object like a floor or a wall, we need to make it immovable. An immovable body will not be pushed or moved by collisions with other dynamic bodies. We can achieve this by calling the setImmovable method on the object's physics body.

© 2025 Zenva Pty Ltd. All rights reserved.

```
ground.body.setImmovable(true);
```

When a body's immovable property is set to true, it will not absorb any momentum from other objects it collides with. Now, when the second ground object hits the first, the first one will remain perfectly still, behaving exactly as a solid ground layer should.

An Efficient Alternative: Static Bodies

While manually setting gravity and immovability works, Phaser provides a more direct and efficient way to create objects that should never move: **static bodies**. Currently, our objects have **dynamic bodies**, which are designed to be moved by the physics system.

A static body, by contrast, has a fixed position and is not affected by gravity, velocity, or collisions from other objects. It serves as an unmovable part of your game world. This is ideal for elements like ground, walls, and platforms.

One way to create a static body is to pass a second argument to the this.physics.add.existing method. This argument is a boolean that, when set to true, specifies that the body should be static.

```
// First, create a regular sprite
const ground = this.add.sprite(180, 400, 'ground');
// Then, add it to the physics system as a static body
this.physics.add.existing(ground, true);

// The other ground object remains dynamic
const ground2 = this.physics.add.sprite(180, 200, 'ground');

// The collider is still needed for interaction
this.physics.add.collider(this.ground, this.ground2);
```

When you use a static body, you no longer need to call setAllowGravity(false) or setImmovable(true), as these properties are inherent to static bodies. If you have physics debugging enabled, you'll also notice a visual difference: dynamic bodies are outlined in pink, while static bodies are outlined in blue.

Performance Benefits

Using static bodies is not just a matter of convenience; it also improves performance. Because static bodies are not expected to move, Phaser's physics engine does not need to perform complex calculations for them, such as acceleration or velocity. A static body has fewer properties than a dynamic body, which reduces the computational overhead on the physics simulation. For games with many platforms or level elements, using static bodies can lead to significant performance gains.

Conclusion

Understanding the difference between static and dynamic bodies is fundamental to using Phaser's arcade physics system effectively. The rule of thumb is simple:

- Use **static bodies** for objects that will never move, such as the ground, walls, and platforms.
- Use **dynamic bodies** for objects that need to move and be affected by physics, such as the player, enemies, and items.

© 2025 Zenva Pty Ltd. All rights reserved.

This approach ensures your game behaves as expected and keeps the physics simulation efficient. In our next lesson, we will apply these concepts to add our player and platform game objects to the scene.

© 2025 Zenva Pty Ltd. All rights reserved.

Welcome back. In the previous lesson, we explored Arcade Physics bodies and their properties. We distinguished between static and dynamic bodies and learned how to enable collisions using a collider. Now, we will apply these concepts by adding our player and platform game objects to the scene.

Refining the Ground Object

For our game, the ground and platforms will be static physics bodies, while the player will be a dynamic body, allowing it to move and jump. Let’s begin by cleaning up the code for our ground object in the create method.

Instead of creating a sprite and then separately adding it to the physics system, we can use the physics object factory to create an Arcade Sprite directly. This factory provides methods that streamline the process. For a static body like our ground, we can use the staticSprite method.

```
const ground = this.physics.add.staticSprite(180, 604, 'ground');
```

The staticSprite method creates a sprite with a static physics body, indicated by the blue outline in debug mode. We have also updated its y-position to place it at the bottom of our scene.

Creating Platforms with Tile Sprites

Our platforms will be constructed from a repeating ‘block’ image. Rather than creating and positioning multiple individual sprites, we can use a more efficient game object called a Tile Sprite. A Tile Sprite is a special type of sprite where you can define a custom width and height, and Phaser will repeat the provided texture to fill those dimensions.

To create our first platform, we will use the add.tileSprite method from our scene’s object factory.

```
const platform = this.add.tileSprite(180, 500, 36 * 4, 30 * 1, 'block');
```

Let’s review the arguments:

- 180 and 500 are the x and y coordinates.
- 36 * 4 and 30 * 1 are the width and height. It is best practice to use multiples of the base image’s dimensions (36×30 pixels in this case) to ensure the texture repeats seamlessly. Here, we are making the platform six blocks wide.
- 'block' is the key for the texture to use.

Since the physics object factory does not have a method to create a tile sprite directly, we first create the game object and then add a physics body to it using physics.add.existing. We pass true as the second argument to make it a static body.

```
this.physics.add.existing(platform, true);
```

Adding the Player

Next, let’s add the player. The player needs a dynamic body to be affected by gravity and to move around. We will use the physics.add.sprite method, which creates a sprite with a dynamic body by

© 2025 Zenva Pty Ltd. All rights reserved.

default. We will also store a reference to the player on our scene class.

```
this.player = this.physics.add.sprite(180, 400, 'player', 3);
```

The player's texture is a spritesheet containing multiple images. The final argument, 3, specifies the frame index, telling Phaser which image from the sheet to display initially. Upon saving, you will see the player appear and fall due to gravity.

Implementing Collisions

To prevent the player from falling through the platform, we need to add a collider. A collider checks for collisions between two physics-enabled objects and produces a physical response.

```
this.physics.add.collider(this.player, platform);
```

With this line, the player will now land on the platform. If we add more platforms, we would need to add more colliders, which can become repetitive.

Using Groups for Efficiency

As our game grows to include multiple platforms, creating a separate collider for each one is inefficient. Phaser provides two better ways to handle this: passing an array of objects to the collider, or using a Phaser Group.

A Phaser Group is an excellent tool for managing a collection of similar game objects. Let's create a group for our platforms and add them to it.

First, create the group and store it as a class property.

```
this.platforms = this.add.group();
```

Next, create your platform game objects as before. For this example, we'll create two.

```
const platform = this.add.tileSprite(180, 500, 36 * 4, 30 * 1, 'block');
this.physics.add.existing(platform, true);

const platform2 = this.add.tileSprite(180, 450, 36 * 4, 30 * 1, 'block');
this.physics.add.existing(platform2, true);
```

Now, add each platform instance to the group.

```
this.platforms.add(platform);
this.platforms.add(platform2);
```

Finally, create a single collider between the player and the entire group.

```
this.physics.add.collider(this.player, this.platforms);
```

Now, any platform added to the this.platforms group will automatically collide with the player,

© 2025 Zenva Pty Ltd. All rights reserved.

simplifying our code significantly.

Finalizing Collisions

To complete our initial setup, we also need to add a collider between the player and the ground.

```
this.physics.add.collider(this.player, this.ground);
```

We now have our player with a dynamic body and our ground and platforms with static bodies, all interacting correctly. In the next lesson, we will bring our player to life by implementing movement controls.

© 2025 Zenva Pty Ltd. All rights reserved.

Welcome back! In this exciting lesson, we will bring our player character to life by giving them some controls. By the end of this article, you will be able to move your character left and right using the arrow keys.

Setting Up Keyboard Input

We will begin by setting up our keyboard input. Phaser makes this straightforward with its cursor keys object. This is a convenient object that helps manage input from the standard arrow keys, as well as the spacebar and shift key. The object contains properties for each of these keys, which we can check to see if a key is being pressed.

We need to create this object and store a reference to it within our scene class. In your create method, add the following line to create the cursor keys and assign them to a new cursors property:

```
// enable cursor keys
this.cursors = this.input.keyboard.createCursorKeys();
```

If you were to log this object to the console (console.log(this.cursors)), you would see that it contains properties for left, right, up, down, space, and shift. Each of these is an object itself, containing information about the key's state, such as a boolean value telling us if it is currently pressed down.

The Update Loop

To constantly check for player input, we need to use the update method. The update method is a core part of a Phaser scene that runs for every single frame of your game. This is the ideal place to check for input and update our player's state accordingly. Let's add the update method to our GameScene class.

```
update() {
  // Game logic goes here
}
```

Implementing Player Movement

Inside the update method, we can now check if one of our arrow keys is being pressed. We will use an if statement to check the isDown property of our left and right cursor keys.

Instead of updating the player's x-coordinate directly, we will use the physics system to move our player. This ensures that our player interacts correctly with other physics-enabled objects. We can control the player's movement by setting their velocity.

- If the left key is pressed, we will set the player's horizontal velocity to a negative value to move left.
- If the right key is pressed, we will set it to a positive value to move right.
- If neither key is pressed, we will set the velocity to zero to stop the player.

Let's add this logic to our update method:

```
update() {
  // movement to the left
  if (this.cursors.left.isDown) {
```

© 2025 Zenva Pty Ltd. All rights reserved.

```
    this.player.body.setVelocityX(-100);
  } else if (this.cursors.right.isDown) {
    // movement to the right
    this.player.body.setVelocityX(100);
  } else {
    // make the player stop
    this.player.body.setVelocityX(0);
  }
}
```

If you save and run the game now, you should be able to move the player left and right with the arrow keys. When you release the keys, the player will stop moving.

Refining the Controls

There is a small issue with our current implementation. If you press both the left and right arrow keys simultaneously, the player will move left because the condition for the left key is checked first. To make the controls feel more intuitive, we should prevent the player from moving if both keys are pressed at the same time.

We can achieve this by modifying our if statements to ensure only one key is being processed at a time. We will check if the left key is down AND the right key is not, and vice-versa.

Update your update method with the following code:

```
update() {
  // movement to the left
  if (this.cursors.left.isDown && !this.cursors.right.isDown) {
    this.player.body.setVelocityX(-100);
  } else if (this.cursors.right.isDown && !this.cursors.left.isDown) {
    // movement to the right
    this.player.body.setVelocityX(100);
  } else {
    // make the player stop
    this.player.body.setVelocityX(0);
  }
}
```

Now, if you try holding both keys, the player will stop moving. Movement will only occur when a single direction key is pressed. This makes the input handling clearer and more robust.

Our player can now move from side to side, but the movement still feels a bit lifeless. In the next lesson, we will bring our player to life with proper animations and by flipping their direction based on their movement. See you there!

© 2025 Zenva Pty Ltd. All rights reserved.

Welcome back. Our player character can now move from side to side, but the movement feels static and lifeless. To improve this, we will add walking animations. We will use the first three frames from our player's spritesheet to create a walking cycle.

Creating the Walking Animation

Animations, much like assets, only need to be defined once for the entire game. The ideal place to do this is in our `LoadingScene`, after all assets have been loaded but before the `GameScene` begins. Let's navigate to the create() method in src/scenes/loading-scene.js to set up our animation.

We will use Phaser's animation manager, accessible via this.anims, to create a new animation. The create() method takes a configuration object that defines the animation's properties.

- **key**: A unique string identifier for the animation. We will name ours 'walking'.
- **frames**: The sequence of frames that make up the animation. We can use the generateFrameNames() utility to create these from our 'player' spritesheet, specifying that we want to use frames 0, 1, and 2.
- **frameRate**: The speed of the animation in frames per second. We'll set this to 12.
- **yoyo**: A boolean that, when true, makes the animation play in reverse after reaching the end. This creates a smooth back-and-forth motion (e.g., 0-1-2-1-0).
- **repeat**: Determines how many times the animation should loop. A value of -1 will make it repeat indefinitely.

Add the following code to the create() method in loading-scene.js:

```
// walking animation
this.anims.create({
  key: 'walking',
  frames: this.anims.generateFrameNames('player', {
    frames: [0, 1, 2]
  }),
  frameRate: 12,
  yoyo: true,
  repeat: -1
});
```

Playing and Stopping the Animation

Now that the animation is defined, we need to trigger it when the player moves. Let's go to the update() method in src/scenes/game-scene.js.

Inside the conditions for left and right movement, we will call the play() method on our player game object. The first argument is the key of the animation we want to play, which is 'walking'.

A common issue is that calling play() on every frame (which happens when a key is held down) will constantly restart the animation, making it appear frozen. To prevent this, we can pass a second argument, true, to the play() method. This tells Phaser to only start the animation if it is not already playing.

```
// movement to the left
if (this.cursors.left.isDown) {
  // ...
  this.player.play('walking', true);
```

© 2025 Zenva Pty Ltd. All rights reserved.

```
} else if (this.cursors.right.isDown) {
  // ...
  this.player.play('walking', true);
}
```

When the player stops moving (in the else block), we need to stop the animation. We can do this with the stop() method. However, this will leave the player on whichever frame the animation was on when it stopped. To provide a consistent appearance, we should also reset the player's sprite to its default, idle frame. We can achieve this using setFrame() and passing the index of the idle frame, which is 3 in our spritesheet.

```
} else {
  // make the player stop
  this.player.body.setVelocityX(0);

  // stop walking animation
  this.player.stop();

  // set default frame
  this.player.setFrame(3);
}
```

Flipping the Player Sprite

Currently, our player always faces left, regardless of the direction of movement. We can fix this by flipping the sprite horizontally when moving to the right. The player game object has a setFlipX() method for this purpose.

When moving left, we'll ensure flipX is set to false (the default orientation). When moving right, we'll set it to true.

```
// movement to the left
if (this.cursors.left.isDown) {
  this.player.body.setVelocityX(-100);
  this.player.setFlipX(false);
  this.player.play('walking', true);
} else if (this.cursors.right.isDown) {
  // movement to the right
  this.player.body.setVelocityX(100);
  this.player.setFlipX(true);
  this.player.play('walking', true);
}
```

Refactoring Player Speed

To make our code cleaner and easier to manage, we should avoid using "magic numbers" like the hard-coded speed value of 100. Let's define a playerSpeed property for our scene and use it instead.

First, add the property in the init() method of GameScene. We'll set the speed to 150.

© 2025 Zenva Pty Ltd. All rights reserved.

```
init() {
  this.playerSpeed = 150;
  // ...
}
```

Next, replace the hard-coded values in the update() method with our new property, this.playerSpeed.

```
// movement to the left
if (this.cursors.left.isDown && !this.cursors.right.isDown) {
  this.player.body.setVelocityX(-this.playerSpeed);
  this.player.setFlipX(false);
  this.player.play('walking', true);
} else if (this.cursors.right.isDown && !this.cursors.left.isDown) {
  // movement to the right
  this.player.body.setVelocityX(this.playerSpeed);
  this.player.setFlipX(true);
  this.player.play('walking', true);
}
```

With these changes, our player now walks smoothly, faces the correct direction, and stops gracefully. In the next lesson, we will build upon this by implementing jumping mechanics.

© 2025 Zenva Pty Ltd. All rights reserved.

Welcome back! In this lesson, we will enhance our player's movement by implementing jumping mechanics. We will also ensure our player is confined to the game world, preventing them from falling off the screen. Currently, our player can move left and right but can exit the screen entirely. Let's address that first.

Containing the Player within the World

To prevent the player from leaving the game area, we can instruct Phaser to make the player's physics body collide with the boundaries of the game world. In your create method, after the player has been created, add the following line:

```
this.player.body.setCollideWorldBounds(true);
```

This single method call tells Phaser to stop this specific game object from moving outside the world's boundaries. By default, these boundaries match the game's dimensions as specified in your game configuration.

However, you may want a level that is larger than the visible screen. In such cases, you can explicitly define the size of the world. To do this, add the following code to your create method:

```
this.physics.world.setBounds(0, 0, 360, 700);
```

The setBounds method defines an invisible rectangular boundary for the physics world. The parameters are:

- The x-coordinate of the top-left corner (0).
- The y-coordinate of the top-left corner (0).
- The width of the world (360).
- The height of the world (700).

With these settings, our world starts at the top-left corner and extends slightly beyond the default height of our game, creating a taller level.

Implementing the Jump

Now, let's add the ability for the player to jump. We will check for input from the up arrow key or the spacebar in our update method. When a jump is initiated, we will apply an upward velocity to the player's physics body.

First, let's define a property for our jump speed in the init method for easy configuration.

```
init() {
    this.playerSpeed = 150;
    this.jumpSpeed = 600;
}
```

Next, we need to adjust our game's gravity to make the jump feel more natural. A low gravity value will cause the player to jump very high and fall slowly. In your main.js file, update the gravity in the physics configuration.

© 2025 Zenva Pty Ltd. All rights reserved.

```
physics: {
    default: 'arcade',
    arcade: {
      gravity: {
        x: 0,
        y: 1000,
      },
      debug: false,
    }
}
```

With the setup complete, let's add the jump logic to the update method, after the left and right movement checks.

```
// handle jumping
if (this.cursors.up.isDown || this.cursors.space.isDown) {
    // give the player a velocity in Y
    this.player.body.setVelocityY(-this.jumpSpeed);

    // stop walking animation
    this.player.stop();

    // set default frame
    this.player.setFrame(2);
}
```

This code checks if the up arrow or spacebar is pressed. If so, it sets a negative Y velocity, causing the player to move upwards. It also stops the walking animation and sets the player's sprite to the jumping frame (index 2).

Preventing Infinite Jumps

If you test the game now, you'll notice a problem: the player can jump infinitely, even in mid-air. To fix this, we must only allow a jump when the player is on the ground.

Phaser's Arcade Physics body provides properties to check for collisions. We can use body.blocked.down and body.touching.down to determine if the player is on a surface. The blocked property checks for collisions with static objects like tiles or the world bounds, while touching checks for collisions with other dynamic physics bodies.

At the beginning of the update method, let's create a constant to store this ground check.

```
update() {
    // are we on the ground?
    const onGround = this.player.body.blocked.down || this.player.body.touching.down;

    // ... rest of the update logic
}
```

© 2025 Zenva Pty Ltd. All rights reserved.

Now, we can modify our jump condition to include this check.

```
// handle jumping
if (onGround && (this.cursors.up.isDown || this.cursors.space.isDown)) {
    // give the player a velocity in Y
    this.player.body.setVelocityY(-this.jumpSpeed);

    // stop walking animation
    this.player.stop();

    // set default frame
    this.player.setFrame(2);
}
```

With this change, the player can only jump when the onGround condition is true, preventing mid-air jumps.

Refining Animations

One final issue remains: if the player starts moving left or right while in the air, the walking animation begins to play. We should only play the walking animation when the player is on the ground.

We can use our onGround constant to fix this. Modify the left and right movement logic as follows:

```
// movement to the left
if (this.cursors.left.isDown && !this.cursors.right.isDown) {
    this.player.body.setVelocityX(-this.playerSpeed);
    this.player.setFlipX(false);

    if (onGround) {
        // play animation if not already playing
        this.player.play('walking', true);
    }
} else if (this.cursors.right.isDown && !this.cursors.left.isDown) {
    // movement to the right
    this.player.body.setVelocityX(this.playerSpeed);
    this.player.setFlipX(true);

    if (onGround) {
        // play animation if not already playing
        this.player.play('walking', true);
    }
} else if (!this.cursors.left.isDown && !this.cursors.right.isDown) {
    // make the player stop
    this.player.body.setVelocityX(0);

    // stop walking animation
    this.player.stop();

    // set default frame
    this.player.setFrame(3);
}
```

© 2025 Zenva Pty Ltd. All rights reserved.

Notice we also changed the final else to an else if. This ensures the idle state (stopping movement and setting the idle frame) only occurs if neither the left nor right keys are being pressed, which is more precise than the previous logic.

With these changes, our basic platformer controller is complete. The player can now walk, stop, and jump, with animations that respond correctly to their actions. In the upcoming lessons, we will continue to build out the level by adding hazards and other interactive objects.

© 2025 Zenva Pty Ltd. All rights reserved.

With your player character now able to walk and jump, it is time to enhance the game world and apply these new mechanics. This challenge focuses on expanding your level to create a more dynamic and interactive environment for the player.

Challenge: Add More Platforms

Your task is to build upon your existing level by adding new platforms. The goal is to test the player's jumping ability and create a more engaging path through the game.

- Add new sets of platforms throughout your level.
- Place these platforms at varying heights and positions to create a fun challenge.
- Ensure the player can reach these new areas by jumping between the various platforms.

Key Implementation Steps

To ensure your new platforms function correctly, there are two critical steps you must follow. These steps will allow the player character to interact with the platforms as solid objects.

1. **Use a Static Physics Body:** For each new platform, you must ensure it has a static physics body. This type of body is essential for stationary objects like ground or platforms, as it will not be affected by gravity or other physics forces.
2. **Add to the Platform Group:** For the player's collision detection to work, you must add each new platform to the "platform" group. Your player's code is already configured to recognize objects in this group as surfaces it can land on.

By completing this challenge, you will not only make your level more interesting but also solidify your understanding of how physics bodies and groups work together to create interactive game environments.

© 2025 Zenva Pty Ltd. All rights reserved.

Welcome back! In the previous lesson, you were issued a challenge to enhance the level by adding new sets of platforms at varying heights and positions. The main goal was to ensure the player could reach these platforms by jumping and that they would function as solid ground. This article will guide you through one possible solution to that challenge.

Adding and Positioning Platforms

To begin, we need to add more platforms to our level. The key is to ensure that each new platform is a static physics body and is added to our “platform” group. This allows the player to collide with them correctly. In this solution, we will ensure our level has a total of three platforms.

After creating the platform instances, we will set their positions to create a vertical path for the player. The following code demonstrates creating three platforms and setting their unique X and Y coordinates.

```
// Platforms Group
this.platforms = this.add.group();

// Platform 1
const platform = this.add.tileSprite(100, 450, 36 * 8, 30 * 1, 'block');
this.physics.add.existing(platform, true);
this.platforms.add(platform);

// Platform 2
const platform2 = this.add.tileSprite(300, 350, 36 * 6, 30 * 1, 'block');
this.physics.add.existing(platform2, true);
this.platforms.add(platform2);

// Platform 3
const platform3 = this.add.tileSprite(100, 150, 36 * 6, 30 * 1, 'block');
this.physics.add.existing(platform3, true);
this.platforms.add(platform3);
```

Instead of having small, single-block platforms, we can extend their width to create longer surfaces for the player to navigate. Since our platforms are tileSprite objects, we can easily adjust their width, and the texture will repeat automatically.

Adjusting the Player’s Start Position

For a final change, we will adjust the player’s starting position. Rather than having the player start on one of the platforms, we will move their starting point to the bottom of the level. This encourages the player to interact with the new platforms immediately by jumping. To do this, we simply update the Y-coordinate provided when creating the player sprite.

```
// In your setupLevel method, where the player is created
this.player = this.physics.add.sprite(180, 500, 'player', 3);
```

After implementing these changes, your level will feature three distinct platforms of different lengths and at different heights. The player will start on the ground floor, ready to jump and climb through the level you have designed.

© 2025 Zenva Pty Ltd. All rights reserved.

Welcome back! In this lesson, we will explore how to make building levels a much more efficient process. By moving our level layout data out of our game logic, we can save time and make it easier to reuse the same code for multiple levels.

Currently, our ground and platforms are placed manually in our code with hard-coded coordinates. This approach is acceptable for initial testing, but it becomes cumbersome when you want to build larger, more complex levels. Adjusting the position of a platform requires guessing new coordinates, reloading the game, and repeating the process until it looks right, which is highly inefficient.

A Quick Tip for Finding Coordinates

To easily find the coordinates for placing game objects, you can use the mouse pointer. By adding a small amount of code to our game, we can listen for a click event on the scene and log the X and Y position of that click. These logged values can then be used to position our platforms and other objects precisely.

As an example, you can add the following code at the bottom of your create method:

```
this.input.on('pointerdown', (pointer) => {
  console.log(pointer.x, pointer.y);
});
```

With this code, whenever you click within the game window, Phaser will log the coordinates to the console, making it simple to find the exact positions you need for your level design.

Refactoring to a Data-Driven Approach

While the coordinate-finding tip is helpful, the underlying issue is the hard-coded values for our platforms and ground. To create a more flexible and reusable system, we should move this data into a separate data model. This model can be an array of objects, where each object defines a game element, its position, and other properties like its size.

Let’s start by defining this data structure in our init method. We will add a new property called levelData to our scene class.

```
init() {
  this.playerSpeed = 150;
  this.jumpSpeed = 600;

  this.levelData = {
    platforms: [
      {
        x: 100,
        y: 450,
        numOfTiles: 8,
        key: 'block'
      },
      {
        x: 300,
        y: 300,
        numOfTiles: 6,
        key: 'block'
      },
```

© 2025 Zenva Pty Ltd. All rights reserved.

```
      {
        x: 100,
        y: 150,
        numOfTiles: 6,
        key: 'block'
      }
    ]
  };
}
```

Here, we've created a levelData object with a platforms property, which is an array. Each object in the array represents one platform, specifying its position (x, y), its length in tiles (numOfTiles), and the asset key to use (key).

Dynamically Creating Game Objects

Now that we have our data model, we can use it to dynamically create the platforms. Let's create a new method called setupLevel to handle the creation of all our level elements.

In this new method, we will first create our group for platforms. Then, we will loop through the platforms array in our levelData and create a tile sprite for each entry.

```
setupLevel() {
  // create all the platforms
  this.platforms = this.add.group();

  this.levelData.platforms.forEach((platform) => {
    const gameObject = this.add.tileSprite(platform.x, platform.y, 36 * platform.numO
fTiles, 30, platform.key);
    this.physics.add.existing(gameObject, true);
    this.platforms.add(gameObject);
  });
}
```

Next, we need to call this new method from our create method. We can now comment out or remove the old, hard-coded platform creation logic.

```
create() {
  // ... old platform code ...

  this.setupLevel();

  // ... rest of create method ...
}
```

With this change, our platforms are now generated based on the data in levelData. You can easily add, remove, or modify platforms just by changing the data array.

Removing Hard-Coded Dimensions

© 2025 Zenva Pty Ltd. All rights reserved.

Our current implementation still has hard-coded width and height values (36 and 30) for the tile sprite. We can make this more robust by dynamically getting these dimensions from the loaded image texture.

We can access Phaser's texture manager to get the dimensions of our asset. Let's update the loop inside setupLevel:

```
setupLevel() {
  this.platforms = this.add.group();

  this.levelData.platforms.forEach((platform) => {
    const { width, height } = this.textures.get(platform.key).get(0);
    const gameObject = this.add.tileSprite(platform.x, platform.y, width * platform.n
umOfTiles, height, platform.key);
    this.physics.add.existing(gameObject, true);
    this.platforms.add(gameObject);
  });
}
```

This code retrieves the texture using its key, gets its first frame, and then extracts the width and height. This ensures that if we change the asset, the dimensions will update automatically.

Extending the Data Model for All Game Objects

We can apply this data-driven approach to all game objects in our level, including the player and the ground. Let's expand our levelData object to include data for them.

```
init() {
  this.playerSpeed = 150;
  this.jumpSpeed = 600;

  this.levelData = {
    platforms: [
      // ... platform data
    ],
    player: {
      x: 180,
      y: 500,
      key: 'player',
      frame: 0
    },
    ground: {
      x: 180,
      y: 604,
      key: 'ground'
    }
  };
}
```

Now, let's move the creation logic for the ground and player into our setupLevel method and use the data from our levelData object. We also need to store a reference to the ground object so our

© 2025 Zenva Pty Ltd. All rights reserved.

colliders can access it.

```
setupLevel() {
  // ... platform creation code ...

  // ground
  this.ground = this.physics.add.staticSprite(this.levelData.ground.x, this.levelData
.ground.y, this.levelData.ground.key);

  // player
  this.player = this.physics.add.sprite(this.levelData.player.x, this.levelData.playe
r.y, this.levelData.player.key, this.levelData.player.frame);
  // constraint player to the game bounds
  this.player.body.setCollideWorldBounds(true);
}
```

Finally, update the create method to remove the old player and ground creation code and ensure the collider references this.ground.

```
create() {
  this.setupLevel();

  // collision detection
  this.physics.add.collider(this.player, this.platforms);
  this.physics.add.collider(this.player, this.ground);

  // ... rest of create method ...
}
```

After removing the old, commented-out code, your create method will be much cleaner. This refactoring makes our code more reusable, robust, and dynamic, setting a strong foundation for creating multiple levels in the future.

In our next lesson, we will take this concept a step further by moving our level data into an external JSON file, which we will then load into our game using Phaser.

© 2025 Zenva Pty Ltd. All rights reserved.

Welcome back. In this lesson, we will move our level data to an external JSON file. This approach will save us time and make it easier to reuse the same game logic for multiple levels, enhancing the scalability and maintainability of our project.

Decoupling Level Data

Currently, our level data is hard-coded directly within our `GameScene`. This means that any change to the level, such as altering platform positions or adding new elements, requires modifying the game's source code. If we wanted to add a second level, we would need to create another large object within the same file. By externalizing this data, we can make our code more dynamic and flexible.

Using JSON for Level Data

We will use a JSON (JavaScript Object Notation) file to store our level data. JSON is a lightweight data-interchange format that is easy for humans to read and write and easy for machines to parse and generate. Its syntax is very similar to JavaScript objects, making it an ideal choice for this purpose.

First, let's create the file to hold our data.

1. In your project's `assets` folder, create a new folder named data.
2. Inside the new data folder, create a file named levelData.json.
3. Copy the levelData object from your GameScene and paste it into levelData.json.

Now, you must convert the JavaScript object syntax to valid JSON format. The key differences are:

- All property keys must be enclosed in double quotes (e.g., "player").
- All string values must also use double quotes.

After converting, your levelData.json file should look similar to this:

```
{
  "player": {
    "x": 190,
    "y": 500,
    "key": "player",
    "frame": 0
  },
  "ground": {
    "x": 180,
    "y": 604,
    "key": "ground"
  },
  "platforms": [
    {
      "x": 100,
      "y": 450,
      "numOfTiles": 8,
      "key": "block"
    },
    {
      "x": 300,
      "y": 300,
      "numOfTiles": 6,
      "key": "block"
```

© 2025 Zenva Pty Ltd. All rights reserved.

```
    },
    {
      "x": 100,
      "y": 150,
      "numOfTiles": 6,
      "key": "block"
    }
  ]
}
```

Loading JSON in Phaser

Before we can use this data, we need to instruct Phaser to load the JSON file. This is done in the preload method of our LoadingScene, alongside our other asset loading calls.

Open src/scenes/loading-scene.js and add the following line to the end of the preload method:

```
// load json data
this.load.json('levelData', 'assets/data/levelData.json');
```

Here, we use the json method from Phaser's loader plugin. The first argument, 'levelData', is the unique key we will use to access this data later. The second argument is the path to the file.

Accessing JSON Data

With the data loaded, we can now access it from our GameScene. Phaser stores all preloaded assets in a cache. We can retrieve our JSON data from this cache.

In src/scenes/game-scene.js, find the hard-coded levelData object and replace it with the following line inside the init method. This will fetch the data from the cache and assign it to the scene's levelData property.

```
init() {
  this.playerSpeed = 150;
  this.jumpSpeed = 600;
  this.levelData = this.cache.json.get('levelData');
}
```

Now, if you run the game, it should function exactly as before, but this time it is reading the level configuration from the external levelData.json file. Any changes made to the JSON file will be reflected in the game upon refreshing.

Updating the Level Layout

Let's modify our level by updating the levelData.json file. We will reposition the platforms and the player to create a more interesting layout.

Update your levelData.json with the following values:

© 2025 Zenva Pty Ltd. All rights reserved.

```
{
  "player": {
    "x": 190,
    "y": 538
  },
  "platforms": [
    {
      "x": 72,
      "y": 450,
      "numOfTiles": 6,
      "key": "block"
    },
    {
      "x": 0,
      "y": 330,
      "numOfTiles": 8,
      "key": "block"
    },
    {
      "x": 72,
      "y": 210,
      "numOfTiles": 8,
      "key": "block"
    },
    {
      "x": 0,
      "y": 90,
      "numOfTiles": 7,
      "key": "block"
    }
  ]
}
```

Adjusting Object Origins

After updating the platform positions, you might notice they are not aligned as expected. This is because a Tile Sprite's default origin is its center (0.5, 0.5). Since we are positioning our platforms based on their top-left corner coordinates, we need to adjust their origin.

In the setupLevel method within GameScene, modify the platform creation loop to set the origin of each platform to its top-left corner.

```
this.levelData.platforms.forEach((platform) => {
  const { width, height } = this.textures.get(platform.key).get(0);
  const gameObject = this.add.tileSprite(platform.x, platform.y, width * platform.num
OfTiles, height, platform.key);
  gameObject.setOrigin(0);
  // ... Rest of loop code
});
```

By calling gameObject.setOrigin(0), we set the origin to (0, 0), which corresponds to the top-left

© 2025 Zenva Pty Ltd. All rights reserved.

corner. Now, when you save and refresh, the platforms will be positioned correctly according to the coordinates in your JSON file.

With this setup, you can now modify your level entirely through the JSON file. This is a significant improvement that makes it much easier to create new levels, share them, or even load them from a server. In the next lesson, we will continue building our level by adding fire hazards.

© 2025 Zenva Pty Ltd. All rights reserved.

Welcome back! In this lesson, we will expand our JSON level data by incorporating new data for our fire hazards and the goal position. This enhancement will ensure our level definition encompasses not just the platforms, but all crucial gameplay elements. The goal position represents the location of the gorilla that the player must reach, while the fire elements will act as hazards for the player to navigate.

Expanding the Level Data

First, let's add the new data to our levelData.json file. We will add two new properties to the main JSON object: goal and fires.

The goal object will define the position and asset for our gorilla. The fires property will be an array of objects, with each object defining a fire hazard's position and asset.

Open your levelData.json file and add the following properties after the platforms array:

```
{
  "player": { ... },
  "ground": { ... },
  "platforms": [ ... ],
  "goal": {
    "x": 30,
    "y": 65,
    "key": "goal"
  },
  "fires": [
    {
      "x": 12.9,
      "y": 538.4,
      "key": "fire"
    },
    {
      "x": 328,
      "y": 188,
      "key": "fire"
    },
    {
      "x": 68.3,
      "y": 307,
      "key": "fire"
    },
    {
      "x": 87.7,
      "y": 427,
      "key": "fire"
    }
  ],
  ...
}
```

With these additions, our JSON file now defines the platforms, player, ground, goal, and initial fire hazards.

© 2025 Zenva Pty Ltd. All rights reserved.

Implementing Fire Hazards

Now, let's update our setupLevel method in GameScene.js to load these new elements. We will start with the fire hazards. Similar to platforms, we will manage them within a group.

First, create a new group for the fires in the setupLevel method.

```
// create all the fire objects
this.fires = this.add.group();
```

Next, we will loop through the fires array from our level data and create a sprite for each entry. Since these hazards should not be affected by gravity or be movable, we will create them as static sprites within the group.

```
this.levelData.fires.forEach((fire) => {
  const gameObject = this.physics.add.staticSprite(fire.x, fire.y, fire.key);
  this.fires.add(gameObject);
});
```

If you run the game now, you might notice the fire sprites are not positioned correctly on the platforms. To fix this, we need to update their origin point to the top-left corner.

```
this.levelData.fires.forEach((fire) => {
  const gameObject = this.physics.add.staticSprite(fire.x, fire.y, fire.key);
  gameObject.setOrigin(0);
  this.fires.add(gameObject);
});
```

After saving, you'll see the sprites are positioned correctly, but their physics bodies are now misaligned. Whenever you manually modify a game object's origin or position, you must also update its physics body. We can do this by calling the reset method on the body.

```
this.levelData.fires.forEach((fire) => {
  const gameObject = this.physics.add.staticSprite(fire.x, fire.y, fire.key);
  gameObject.setOrigin(0);
  gameObject.body.reset();
  this.fires.add(gameObject);
});
```

You may need to adjust the y-coordinate of the first fire object to ensure it sits correctly on the ground. Let's update it in levelData.json.

```
"fires": [
    {
      "x": 12.9,
      "y": 545,
      "key": "fire"
```

© 2025 Zenva Pty Ltd. All rights reserved.

```
    },
...
]
```

Animating the Fire

Our fire sprites are static, but we can bring them to life with an animation. The fire spritesheet contains two frames that we can cycle through to create a flickering effect.

Navigate to LoadingScene.js and add a new animation in the create method. We will call it ‘burning’.

```
// fire animation
this.anims.create({
  key: 'burning',
  frames: this.anims.generateFrameNames('fire', {
    frames: [0, 1]
  }),
  frameRate: 4,
  repeat: -1
});
```

Now, return to GameScene.js. Inside the loop where we create the fire objects, call the play method to start the ‘burning’ animation.

```
this.levelData.fires.forEach((fire) => {
  // Existing code

  // play burning animation
  gameObject.play('burning');
});
```

Save your changes, and you will see the fire hazards flickering in your game, just like classic arcade flames.

A Level Design Trick

Manually finding the exact x and y coordinates for each object can be tedious. Here is a development trick to make level design easier. We can make our game objects draggable, move them to the desired position during gameplay, and log their final coordinates to the console. You can then copy these values into your JSON file.

To enable this, we first need to make our game objects interactive. In the forEach loop for the fires, call the setInteractive method on each game object. To enable dragging, we pass a configuration object.

```
// for level creation
gameObject.setInteractive({
  draggable: true,
});
```

© 2025 Zenva Pty Ltd. All rights reserved.

Next, we need to listen for the ‘drag’ event on Phaser’s global input manager. Add this listener outside of the loop, as it only needs to be set up once.

```
// for level creation
this.input.on('drag', (pointer, gameObject, dragX, dragY) => {
  gameObject.x = dragX;
  gameObject.y = dragY;
  console.log(dragX, dragY);
});
```

Now, when you run the game, you can click and drag the fire sprites. Their new x and y coordinates will be logged to the developer console. Remember, this functionality is for development purposes only and should be removed or disabled before you publish your game.

Adding the Goal

Finally, let’s add the goal object—the gorilla—to the top of our level. The process is similar to how we added the player.

In the setupLevel method of GameScene.js, add the code to create the goal as a static sprite, using the data from our JSON file.

```
// goal
this.goal = this.physics.add.staticSprite(this.levelData.goal.x, this.levelData.goal.
y, this.levelData.goal.key);
```

After saving, the gorilla will appear at the top of the level, waiting for the player.

This is a significant step forward. Our level data is now more flexible and powerful, defining multiple types of game objects. In the next lesson, we will make the fire hazards dangerous by detecting when the player overlaps with them.

© 2025 Zenva Pty Ltd. All rights reserved.

Welcome back. In this lesson, we will implement the logic to detect when our player character overlaps with hazards or the goal. This will allow us to restart the level when the player touches a fire object or reaches the goal.

Colliders vs. Overlaps

Currently, when our player interacts with the fire or goal game objects, nothing happens. To change this, we need to add what is known as an overlap check. An overlap is similar to a collider, but with one key difference. In Phaser's Arcade Physics system, a collider creates a physical collision that stops movement, such as the interaction between our player and the platforms. An overlap, on the other hand, simply detects the contact between two game objects without physically blocking their movement. For our fire and goal objects, we want the player to be able to touch them, and upon contact, we will reset the level.

Adding an Overlap Check

To add an overlap check, we will use the Arcade Physics system in a way that is very similar to how we created our colliders. In the create method of your GameScene, we will add the following code to detect an overlap between the player and the fire hazards.

```
this.physics.add.overlap(this.player, this.fires, this.restartGame, null, this);
```

The overlap method takes several arguments:

- The first argument is the first game object or group for the check, in this case, this.player.
- The second argument is the second game object or group, which is our this.fires group.
- The third argument is a callback function that will be invoked when the overlap occurs. We will create a new method named restartGame for this.
- The fourth argument is an optional "process callback". This function runs before the main callback and allows for additional logic to determine if the overlap should be processed. We do not need this, so we will pass null.
- The final argument is the scope or context in which the callback is executed. We will pass this to refer to the current scene.

The Restart Callback Method

Now, let's create the restartGame method at the bottom of our GameScene class. Initially, we will add a simple console log to confirm that the overlap is being detected correctly.

```
restartGame() {
  console.log('test');
}
```

If you save and run the game, you will notice that as soon as the player touches a fire object, the message "test" is logged to the console multiple times. This is because the overlap check runs on every frame of the game loop. To ensure our restart logic only runs once, we need to disable the interaction after the first overlap. We can achieve this by disabling the player's physics body.

```
restartGame() {
  // disable player physics body
  this.player.body.enable = false;
```

© 2025 Zenva Pty Ltd. All rights reserved.

```
}
```

With this change, when the player overlaps with a fire, their physics body is disabled. This stops the player from moving and also prevents the overlap event from firing repeatedly.

Restarting the Scene with a Fade Effect

Instead of just stopping the player, we want to restart the entire scene. To create a smoother transition, we will have the camera fade out before the scene restarts. We can trigger a fade effect on the main camera.

```
this.cameras.main.fade(500);
```

This line will start a 500-millisecond fade-to-black effect. Next, we need to wait for this effect to complete before we restart the scene. We can do this by listening for the camerafadeoutcomplete event on the camera.

Let’s update our restartGame method to include the fade effect and the scene restart logic.

```
restartGame() {
  // disable player physics body
  this.player.body.enable = false;

  // fade out
  this.cameras.main.fade(500);

  // when fade out completes, restart scene
  this.cameras.main.once('camerafadeoutcomplete', () => {
    // restart the scene
    this.scene.restart();
  });
}
```

Now, when the player touches a fire, the screen will fade out, and then the level will restart from the beginning.

Handling the Goal

Finally, we need to apply the same logic for when the player reaches the goal. We can do this by adding another overlap check in our create method, this time between the player and the goal object.

```
this.physics.add.overlap(this.player, this.goal, this.restartGame, null, this);
```

Since we are using the same restartGame method as the callback, reaching the goal will now also trigger the fade-out and restart the level. With this, our core game loop is complete. You can now play the level from start to finish.

© 2025 Zenva Pty Ltd. All rights reserved.

Welcome back. In this lesson, we will optimize our game by transitioning from regular Phaser groups to Physics Groups. This change will not only streamline our code but also leverage Phaser's internal optimizations for better performance.

Using Physics Groups for Optimization

Currently, we are using standard Phaser groups for our platforms and fire hazards. We then manually add physics properties to the objects within them. A more efficient approach is to use Physics Groups. When you create a game object using a Physics Group, Phaser automatically adds the appropriate physics body for that group type. This also allows Phaser to use internal logic that makes collision detection more efficient.

Converting Platforms to a Static Group

Let's begin by converting our platforms group. We will change it from a regular group to a **Static Group**. A static group ensures that any game object created from it will automatically have a static physics body, meaning it won't be affected by gravity or other forces.

In the setupLevel method, find the line where the platforms group is initialized and modify it as follows:

```
// create all the platforms
this.platforms = this.physics.add.staticGroup();
```

With this change, we no longer need to manually add a physics body to each platform tile sprite. When we add a game object to this static group, Phaser handles the creation of its static physics body automatically. If you run the game, you'll see that the platforms behave exactly as before, but our code is now more efficient.

Converting Fires to a Physics Group

Next, we will do the same for our fires group. However, instead of a static group, we will use a regular Arcade Physics group. This provides us with the flexibility to potentially make these hazards move in the future.

Modify the initialization of the fires group:

```
// create all the fire objects
this.fires = this.physics.add.group({
  allowGravity: false,
  immovable: true
});
```

If you were to only change this.add.group() to this.physics.add.group(), you would notice the fire objects immediately fall off the screen. This is because a standard physics group assigns a dynamic body to its members, which is affected by gravity. To prevent this, we pass a configuration object:

- allowGravity: false: This property prevents gravity from affecting the objects in this group.
- immovable: true: This makes the objects immovable, so they won't be pushed around by other physics bodies like the player.

© 2025 Zenva Pty Ltd. All rights reserved.

Refactoring Object Creation

Our current method for creating fire objects is now redundant. We create a static sprite and then add it to a group that overrides its physics properties. We can simplify this by using the group's own create() method, which creates a game object and adds it to the group in a single step.

Let's refactor the loop that creates the fire objects:

```
this.levelData.fires.forEach((fire) => {
  const gameObject = this.fires.create(fire.x, fire.y, fire.key);
  gameObject.setOrigin(0);

  // play burning animation
  gameObject.play('burning');
});
```

This code is much cleaner. The this.fires.create() method handles the creation and addition of the game object to the physics group. Furthermore, because we are using a physics group, Phaser will automatically adjust the physics body's size and offset if we change the game object's origin, simplifying our code even further.

Implementing Camera Follow

To enhance the gameplay experience, we will make the camera follow the player throughout the level. This requires two steps: setting the camera's boundaries and telling it which object to follow.

At the end of your setupLevel method, add the following code:

```
// camera bounds
this.cameras.main.setBounds(0, 0, 360, 700);
this.cameras.main.startFollow(this.player);
```

Here is a breakdown of what this code does:

1. this.cameras.main.setBounds(...): This method restricts the camera's view to the dimensions of our game world.
2. this.cameras.main.startFollow(this.player): This tells the main camera to lock onto and follow the player game object.

Now, when you run the game, the camera will move to keep the player centered in the view as they navigate the level.

We have successfully optimized our groups and improved the camera behavior. In the next lesson, we will continue to expand our gameplay by adding barrels that the player must dodge.

© 2025 Zenva Pty Ltd. All rights reserved.

Welcome back! In this lesson, we will increase the challenge of our game by having the gorilla throw barrels down at the player. These barrels will spawn periodically, roll across the level, and the player will need to avoid them to survive. We will define the barrel's behavior, set up a spawner using a timer, and handle all the necessary collision logic.

Configuring Barrel and World Properties

First, we need to define several properties for our barrels: how often they spawn, their speed, and how long they last before disappearing. We will also define our world's dimensions to avoid hardcoding values. Let's add these settings to our levelData.json file.

Open assets/data/levelData.json and add a new spawner object and a world object at the end of the file.

```
{
  "player": { ... },
  "ground": { ... },
  "platforms": [ ... ],
  "goal": { ... },
  "fires": [ ... ],
  "spawner": {
    "interval": 2000,
    "speed": 100,
    "lifespan": 3000
  },
  "world": {
    "width": 360,
    "height": 700
  }
}
```

With these new values in our JSON file, let's update our GameScene to use them. In src/scenes/game-scene.js, copy and remove the world bounds and camera follow code from create(). Navigate to the setupLevel method and paste in these configurations here. We will replace the hardcoded numbers for the world and camera bounds with the data we just defined.

```
setupLevel() {
  // world bounds
  this.physics.world.setBounds(0, 0, this.levelData.world.width, this.levelData.world
.height);

  // ... create platforms ...
  // ... create ground ...
  // ... create player ...
  // ... create fires ...
  // ... create goal ...

  // camera bounds
  this.cameras.main.setBounds(0, 0, this.levelData.world.width, this.levelData.world.
height);
  this.cameras.main.startFollow(this.player);
}
```

© 2025 Zenva Pty Ltd. All rights reserved.

Setting Up the Barrel Spawner

Now, let's create the logic for spawning barrels. We will create a new method in our GameScene class called setupSpawner. This method will contain a physics group for our barrels and a timed event to create them periodically.

1. First, define the new method at the bottom of the GameScene class.
2. Inside it, create a physics group for the barrels. We will store this group in a class property called this.barrels. We'll also ensure that any barrel created in this group will collide with the world bounds.
3. Next, we will use Phaser's Time Plugin to create a timed event. This allows us to execute code after a specified delay. We will configure it to loop, using the interval from our levelData.json file.
4. The callback function for this event will create a new barrel at the gorilla's position.

Add the following method to game-scene.js:

```
setupSpawner() {
  // barrel group
  this.barrels = this.physics.add.group({
    collideWorldBounds: true,
  });

  this.time.addEvent({
    delay: this.levelData.spawner.interval,
    loop: true,
    callback: () => {
      // create a barrel
      this.barrels.create(this.goal.x, this.goal.y, 'barrel');
    }
  })
}
```

To activate the spawner, we must call this new method from our scene's create method. Add the call after setupLevel.

```
create() {
  // add all level elements
  this.setupLevel();
  // initiate barrel spawner
  this.setupSpawner();

  // ...
}
```

Handling Barrel Collisions

© 2025 Zenva Pty Ltd. All rights reserved.

Currently, our newly spawned barrels will fall through the platforms. We need to add collision detection between the barrels and the level geometry. We can update our existing collider to include the barrels group.

In the create method, modify the collider to handle both the player and the barrels colliding with the platforms and the ground.

```
create() {
  // ...
  this.setupSpawner();

  // collision detection
  this.physics.add.collider([this.player, this.barrels], [this.platforms, this.ground
]);
  // ...
}
```

If you had a separate collider for the player and the ground, you can now remove or comment it out, as the new line handles both.

Managing Barrel Lifespan

To prevent our game from being flooded with barrels, we will make them disappear after a certain amount of time. We can achieve this by adding another timed event inside our spawner's callback, which will trigger once for each barrel created.

Update the setupSpawner method as follows:

```
setupSpawner() {
  // ...
  this.time.addEvent({
    delay: this.levelData.spawner.interval,
    loop: true,
    callback: () => {
      // create a barrel
      const barrel = this.barrels.create(this.goal.x, this.goal.y, 'barrel');

      // lifespan
      this.time.addEvent({
        delay: this.levelData.spawner.lifespan,
        callback: () => {
          barrel.destroy();
        }
      })
    }
  })
}
```

Adding Movement and Bounce

© 2025 Zenva Pty Ltd. All rights reserved.

Our barrels now spawn and disappear, but they don't move horizontally. Let's give them some velocity. We also want them to bounce off the walls and platforms realistically.

First, let's set their horizontal velocity when they are created. We will use the speed property from our JSON data. While we're at it, the current lifespan of 3 seconds is too short for a moving barrel. Let's update it in levelData.json from 3000 to 30000.

```
// In levelData.json
"lifespan": 30000
```

Next, to make the barrels bounce, we will set the bounceX and bounceY properties on the physics group. A value of 1 for bounceX will cause a full rebound off the vertical walls, while a small value for bounceY will create a slight bounce on platforms.

Update the setupSpawner method with these changes:

```
setupSpawner() {
  // barrel group
  this.barrels = this.physics.add.group({
    collideWorldBounds: true,
    bounceX: 1,
    bounceY: 0.1,
  });

  this.time.addEvent({
    delay: this.levelData.spawner.interval,
    loop: true,
    callback: () => {
      // create a barrel
      const barrel = this.barrels.create(this.goal.x, this.goal.y, 'barrel');

      // set properties
      barrel.setVelocityX(this.levelData.spawner.speed);

      // lifespan
      this.time.addEvent({
        delay: this.levelData.spawner.lifespan,
        callback: () => {
          barrel.destroy();
        }
      })
    }
  })
}
```

Player and Barrel Interaction

Finally, we need to make the barrels dangerous to the player. If the player touches a barrel, the game should restart. We can accomplish this by adding the barrels group to our overlap check in the

© 2025 Zenva Pty Ltd. All rights reserved.

create method.

Modify the overlap check to include an array of objects that will trigger a game restart.

```
create() {
  // ...
  this.physics.add.collider([this.player, this.barrels], [this.platforms, this.ground
]);
  // overlap checks
  this.physics.add.overlap(this.player, [this.fires, this.goal, this.barrels], this.r
estartGame, null, this);

  // enable cursor keys
  this.cursors = this.input.keyboard.createCursorKeys();
}
```

Now, if the player overlaps with a fire object or a barrel, the restartGame method will be called. Our barrels are now fully functional, spawning, rolling across the level, and posing a threat to the player.

In our current implementation, we are constantly creating and destroying barrel objects. While this works, it is not the most efficient approach. In the next lesson, we will explore object pooling to reuse barrels, improving our game’s performance.

© 2025 Zenva Pty Ltd. All rights reserved.

Before we move on to our next lesson, here is a challenge for you. Currently, when we spawn barrels in our game, they all have the same properties. They share the same speed, direction, and lifespan. To make our game a little more challenging, we will add some randomness to these properties when the objects are spawned.

The Challenge

For this challenge, you will need to modify the barrel spawning logic to achieve the following for each barrel:

- Randomize the speed of each barrel when it spawns.
- Randomize the bounce direction, so the barrels will move either left or right.

Hint

In Phaser, we can generate random numbers by using the built-in math utilities. The Phaser.Math.Between() method can be used to generate a random integer between a minimum and maximum value, inclusive. You can provide a minimum and maximum value to generate a number between those two values.

```
Phaser.Math.Between(min, max)
```

© 2025 Zenva Pty Ltd. All rights reserved.

Welcome back. In this lesson, we will address the challenge from the previous session: randomizing both the speed and the initial direction of the barrels as they spawn. This will add a layer of unpredictability to the game, making it more engaging. To achieve this, we will modify the barrel's velocity property upon creation.

Randomizing Barrel Speed

First, let's focus on giving each barrel a random speed. Currently, all barrels spawn with the same speed defined in our JSON file. To create a range of possible speeds, we will update our levelData.json file. We will change the speed property from a single value to an object containing minimum and maximum values.

In assets/data/levelData.json, find the spawner object and modify the speed property as follows:

```
"spawner": {
  "interval": 2000,
  "speed": {
    "min": 90,
    "max": 120
  },
  "lifespan": 30000
},
```

With this change, we have defined a speed range between 90 and 120. Now, we can use these values in our GameScene.js file to calculate a random speed for each barrel.

Inside the callback function of our spawner's timed event, we will use Phaser's built-in math utility, Phaser.Math.Between. This function returns a random integer between the two specified values, inclusive. We will create a new variable called speed to store this random value.

```
// set properties
const speed = Phaser.Math.Between(this.levelData.spawner.speed.min, this.levelData.sp
awner.speed.max);
```

Randomizing Barrel Direction

The second part of the challenge is to make barrels move either left or right. We can control the horizontal direction by setting the barrel's X-velocity to either a positive value (for moving right) or a negative value (for moving left). To randomize this, we can generate a random number that represents the direction.

We will create a new variable called direction. We can use Phaser.Math.Between(0, 1) to get a random result of either 0 or 1. We can then map these values to our desired direction multipliers: 1 for right and -1 for left.

A concise way to do this is with a ternary operator. If the random number is 0, we set direction to -1; otherwise, we set it to 1.

```
const direction = Phaser.Math.Between(0, 1) === 0 ? -1 : 1;
```

© 2025 Zenva Pty Ltd. All rights reserved.

Applying the Random Values

Now that we have a random speed and a random direction, we can combine them to set the barrel's horizontal velocity. We simply multiply the speed by the direction value. This will result in a positive velocity when direction is 1 and a negative velocity when direction is -1.

We apply this result using the setVelocityX method on the barrel.

```
barrel.setVelocityX(speed * direction);
```

Your spawner's callback function in setupSpawner should now contain this new logic, making each barrel's movement unique.

```
callback: () => {
  // create a barrel
  const barrel = this.barrels.create(this.goal.x, this.goal.y, 'barrel');

  // set properties
  const speed = Phaser.Math.Between(this.levelData.spawner.speed.min, this.levelData.
spawner.speed.max);
  const direction = Phaser.Math.Between(0, 1) === 0 ? -1 : 1;
  barrel.setVelocityX(speed * direction);

  // ... rest of the function
}
```

With these changes implemented, if you run the game, you will see barrels spawning at different speeds and moving in random directions, successfully completing the challenge.

© 2025 Zenva Pty Ltd. All rights reserved.

Welcome back. In this lesson, we will explore an important concept in game development known as object pooling. Object pooling is a design pattern focused on reusing objects instead of constantly creating and destroying them. This approach is particularly beneficial for performance, helping your game run smoothly and efficiently, especially when generating numerous objects like the barrels in our game.

Why is Object Pooling Important?

In JavaScript, the process of creating and destroying objects involves a mechanism called the garbage collector. The garbage collector is responsible for freeing up memory that is no longer in use. However, you do not have direct control over when it runs. When it does activate, it can cause a noticeable stutter or lag in your game, especially if it has a lot of memory to process. By reusing objects, you minimize the work for the garbage collector, which helps maintain a smooth and consistent frame rate.

Fortunately, Phaser makes object pooling straightforward. Groups in Phaser come with built-in support for pooling, which means we do not need to create a custom system from scratch.

Implementing Object Pooling

Let’s update our barrel generation code to utilize an object pool. Previously, we created a new barrel each time one was needed and destroyed it after its lifespan expired. We will now modify this logic to reuse barrels instead.

Using the `get` Method

Instead of using the create() method on our group, we will use the get() method. This method is designed for pooling and performs two key actions:

- First, it checks the group for an inactive game object.
- If an inactive object is found, Phaser returns it for reuse.
- If no inactive objects are available, it creates a new one.

In the setupSpawner method, locate the line where we create the barrel and change create to get. The get() method accepts the same arguments, allowing you to set the initial position and asset key.

```
// create a barrel
const barrel = this.barrels.get(this.goal.x, this.goal.y, 'barrel');
```

Deactivating Game Objects

For an object to be reused, it must be made inactive rather than destroyed. We will modify the timer event that handles the barrel’s lifespan. Instead of calling destroy() on the barrel, we will use the group’s killAndHide() method. This method deactivates the game object and hides it from the scene, making it available for the pool.

```
// lifespan
this.time.addEvent({
  delay: this.levelData.spawner.lifespan,
  callback: () => {
    this.barrels.killAndHide(barrel);
  }
})
```

© 2025 Zenva Pty Ltd. All rights reserved.

Handling Physics Bodies

After implementing killAndHide(), you might notice that while the barrel sprite disappears, its physics body continues to move. This is because deactivating the game object does not automatically disable its physics body. We must do this manually.

Immediately after calling killAndHide(), add a line to disable the barrel's physics body.

```
this.time.addEvent({
  delay: this.levelData.spawner.lifespan,
  callback: () => {
    this.barrels.killAndHide(barrel);
    barrel.body.enable = false;
  }
})
```

Reactivating Game Objects

Another issue arises when we reuse an object: it remains inactive and invisible. When the get() method retrieves an inactive object from the pool, we need to explicitly reactivate it.

After getting a barrel from the group, we must perform the following steps:

1. Make the object active again using setActive(true).
2. Make the object visible again using setVisible(true).
3. Re-enable its physics body by setting body.enable to true.

Update your barrel creation logic to include these steps.

```
// create a barrel
const barrel = this.barrels.get(this.goal.x, this.goal.y, 'barrel');
barrel.setActive(true).setVisible(true);
barrel.body.enable = true;
```

Verifying the Pool

To confirm that our object pool is working correctly, we can log the number of children in our barrels group. The count should increase initially and then stabilize once the pool starts reusing barrels, rather than growing indefinitely.

Add a console.log statement after creating and setting up a barrel.

```
console.log(this.barrels.getChildren().length);
```

Run the game and observe the browser's console. You will see the number of barrels increase to a certain point and then hold steady, proving that objects are being recycled.

After testing, you can remove the console.log statement and revert any temporary changes made to

© 2025 Zenva Pty Ltd. All rights reserved.

your levelData.json file, such as shortened lifespan or interval values.

You have now successfully implemented object pooling to efficiently manage and reuse objects in your game, contributing to better performance and a smoother player experience.

© 2025 Zenva Pty Ltd. All rights reserved.

Congratulations on making it to the end of the course! It has been a pleasure guiding you through your journey into Phaser 4 game development. Over the past lessons, we have explored Phaser 4, a modern, open-source JavaScript framework that allows us to build fully interactive 2D browser games from the ground up. Along the way, you have acquired essential game development skills and built a Mario-style platformer you can be proud of.

Why Phaser 4 is an Excellent Choice

Phaser 4 was the perfect tool for this project for several key reasons, making it a fantastic choice for aspiring and experienced developers alike.

- **Cross-Platform Compatibility:** Your game runs directly in any modern web browser, reaching a wide audience without extra effort.
- **Open-Source and Free:** With no licensing restrictions, a great community, and complete freedom, you can create and distribute your games without barriers.
- **Versatile for 2D Games:** It is ideal for a wide range of genres, including platformers, arcade games, and casual gameplay.
- **Modern JavaScript:** Phaser is built with modern ES6+ features that make development clean, efficient, and powerful.
- **Rich Feature Set:** Phaser has built-in systems for physics, animations, tilemaps, input handling, scenes, and object pooling, providing all the tools you need.

Your Mario-Style Platformer Project

Together, we built a classic-inspired game where players navigate levels, avoid hazards, and interact with objects like barrels and collectibles. In creating this project, you implemented a variety of core game mechanics.

- Core mechanics like walking and jumping for the player character.
- Detecting collisions with platforms, hazards, and other objects.
- Animating sprites for smooth and believable movement.
- Implementing dynamic and static objects, including barrels that bounce and move.
- Using object pools for performance optimization.
- Creating a playable game loop with clear win-loss conditions.

You have successfully created a fully playable platformer and hopefully gained ideas for how to expand it with new levels, power-ups, or enemies.

What You Accomplished

Let’s recap the key technical skills and concepts we mastered together. You not only know how to build these features but also understand why they work and how to adapt them for your own future projects.

1. We mastered Phaser’s Arcade Physics system to handle gravity, movement, and collisions.
2. We implemented collision and overlap checks for players, platforms, and interactive objects.
3. We built classic platformer mechanics like walking, jumping, and blocking against world bounds.
4. We created timed spawners to generate barrels and other obstacles at set intervals.
5. We used object pooling to efficiently recycle game objects like barrels and optimize performance.
6. We loaded level data from JSON files, allowing you to design and expand your game world with ease.

Your Journey Continues

© 2025 Zenva Pty Ltd. All rights reserved.

Thank you again for joining this course. We hope you enjoyed building your platformer and feel inspired to keep creating with Phaser. Do not stop here! We encourage you to experiment with what you have learned, explore new mechanics, and continue growing your skills.

Zenva is an online learning academy with over a million students, featuring a wide range of courses for those who are just starting out or for developers who want to learn something new. Our courses are versatile, allowing you to learn in the way that suits you best, whether through video tutorials, lesson summaries, or by following along with the included project files.

Until next time, happy coding and keep building amazing games!

© 2025 Zenva Pty Ltd. All rights reserved.

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

index.html

Found in folder: **/**

This HTML file sets up the structure for a web page that will run a Phaser game. It includes the Phaser library and a main JavaScript file, which will contain the game logic. The page is set to have no margins and the game canvas will fill the entire width of the page.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <style>
    body {
      margin: 0;
    }
    canvas {
      width: 100%;
    }
  </style>
</head>
<body>
  <script src="src/lib/phaser.min.js"></script>
  <script src="src/main.js" type="module"></script>
</body>
</html>
```

main.js

Found in folder: **/src**

This code imports the necessary game scenes, defines the game’s configuration, and creates a new Phaser game instance using the specified configuration. The configuration includes settings for the game’s type, scenes, title, pixel art, scaling, and physics. The scaling is set to a width of 360 and a height of 640, with the game centered and scaled to fit the screen, while the physics use the arcade system with a gravity of 1000 in the y-direction.

```
import { GameScene } from './scenes/game-scene.js';
import { LoadingScene } from './scenes/loading-scene.js';

// our game's configuration
const config = {
  type: Phaser.AUTO,
  scene: [LoadingScene, GameScene],
  title: 'Monster Kong',
  pixelArt: false,
```

© 2025 Zenva Pty Ltd. All rights reserved.

```
  scale: {
    width: 360,
    height: 640,
    autoCenter: Phaser.Scale.CENTER_BOTH,
    mode: Phaser.Scale.FIT,
  },
  physics: {
    default: 'arcade',
    arcade: {
      gravity: {
        x: 0,
        y: 1000,
      },
      debug: false,
    }
  }
};

// create the game, and pass it the configuration
const game = new Phaser.Game(config);
```

game-scene.js

Found in folder: **/src/scenes**

This code defines a GameScene class that extends Phaser.Scene. The class initializes game parameters, sets up the level elements, and handles player movement and collision detection. It also includes a barrel spawner that periodically generates barrels with random speeds and directions.

```
export class GameScene extends Phaser.Scene {
  constructor() {
    super({ key: 'GameScene' });
  }

  // some parameters for our scene
  init() {
    this.playerSpeed = 150;
    this.jumpSpeed = 600;
    this.levelData = this.cache.json.get('levelData');
  }

  // executed once, after assets were loaded
  create() {
    // add all level elements
    this.setupLevel();
    // initiate barrel spawner
    this.setupSpawner();

    // collision detection
    this.physics.add.collider([this.player, this.barrels], [this.platforms, this.grou
nd]);
    // overlap checks
    this.physics.add.overlap(this.player, this.goal, this.restartGame, null, this);
```

© 2025 Zenva Pty Ltd. All rights reserved.

```
    // enable cursor keys
    this.cursors = this.input.keyboard.createCursorKeys();

    // this.input.on('pointerdown', (pointer) => {
    //   console.log(pointer.x, pointer.y);
    // });
  }

  update() {
    // are we on the ground?
    const onGround = this.player.body.blocked.down || this.player.body.touching.down;

    // movement to the left
    if (this.cursors.left.isDown && !this.cursors.right.isDown) {
      this.player.body.setVelocityX(-this.playerSpeed);

      this.player.setFlipX(false);

      if (onGround) {
        // play animation if not already playing
        this.player.play('walking', true);
      }
    } else if (this.cursors.right.isDown && !this.cursors.left.isDown) {
      // movement to the right
      this.player.body.setVelocityX(this.playerSpeed);

      this.player.setFlipX(true);

      if (onGround) {
        // play animation if not already playing
        this.player.play('walking', true);
      }
    } else if (!this.cursors.left.isDown && !this.cursors.right.isDown) {
      // make the player stop
      this.player.body.setVelocityX(0);

      // stop walking animation
      this.player.stop();

      // set default frame
      this.player.setFrame(3);
    }

    // handle jumping
    if (onGround && (this.cursors.up.isDown || this.cursors.space.isDown)) {
      // give the player a velocity in Y
      this.player.body.setVelocityY(-this.jumpSpeed);

      // stop walking animation
      this.player.stop();

      // set default frame
      this.player.setFrame(2);
    }
```

© 2025 Zenva Pty Ltd. All rights reserved.

```
  }

  setupLevel() {
    // world bounds
    this.physics.world.setBounds(0, 0, this.levelData.world.width, this.levelData.wor
ld.height);

    // create all the platforms
    this.platforms = this.physics.add.staticGroup();

    this.levelData.platforms.forEach((platform) => {
      const { width, height } = this.textures.get(platform.key).get(0);
      const gameObject = this.add.tileSprite(platform.x, platform.y, width * platform
.numOfTiles, height, platform.key);
      gameObject.setOrigin(0);
      // this.physics.add.existing(gameObject, true);
      this.platforms.add(gameObject);
    });

    // ground
    this.ground = this.physics.add.staticSprite(this.levelData.ground.x, this.levelDa
ta.ground.y, this.levelData.ground.key);

    // player
    this.player = this.physics.add.sprite(this.levelData.player.x, this.levelData.pla
yer.y, this.levelData.player.key, this.levelData.player.frame);
    // constraint player to the game bounds
    this.player.body.setCollideWorldBounds(true);

    // create all the fire objects
    this.fires = this.physics.add.group({
      allowGravity: false,
      immovable: true
    });
    this.levelData.fires.forEach((fire) => {
      const gameObject = this.fires.create(fire.x, fire.y, fire.key);
      gameObject.setOrigin(0);

      // play burning animation
      gameObject.play('burning');

      // // for level creation
      // gameObject.setInteractive({
      //   draggable: true,
      // });
    });

    // // for level creation
    // this.input.on('drag', (pointer, gameObject, dragX, dragY) => {
    //   gameObject.x = dragX;
    //   gameObject.y = dragY;
    //   console.log(dragX, dragY);
    // });

    // goal
```

© 2025 Zenva Pty Ltd. All rights reserved.

```
    this.goal = this.physics.add.staticSprite(this.levelData.goal.x, this.levelData.g
oal.y, this.levelData.goal.key);

    // camera bounds
    this.cameras.main.setBounds(0, 0, this.levelData.world.width, this.levelData.worl
d.height);
    this.cameras.main.startFollow(this.player);
  }

  // restart game (game over + you won!)
  restartGame() {
    // disable player physics body
    this.player.body.enable = false;

    // fade out
    this.cameras.main.fade(500);
    // when fade out completes, restart scene
    this.cameras.main.once('camerafadeoutcomplete', () => {
      // restart the scene
      this.scene.restart();
    });
  }

  // generation of barrels
  setupSpawner() {
    // barrel group
    this.barrels = this.physics.add.group({
      collideWorldBounds: true,
      bounceX: 1,
      bounceY: 0.1,
    });

    this.time.addEvent({
      delay: this.levelData.spawner.interval,
      loop: true,
      callback: () => {
        // create a barrel
        const barrel = this.barrels.get(this.goal.x, this.goal.y, 'barrel');
        barrel.setActive(true).setVisible(true);
        barrel.body.enable = true;

        // set properties
        const speed = Phaser.Math.Between(this.levelData.spawner.speed.min, this.leve
lData.spawner.speed.max);
        const direction = Phaser.Math.Between(0, 1) === 0 ? -1 : 1;
        barrel.setVelocityX(speed * direction);

        //console.log(this.barrels.getChildren().length);

        // lifespan
        this.time.addEvent({
          delay: this.levelData.spawner.lifespan,
          callback: () => {
            this.barrels.killAndHide(barrel);
            barrel.body.enable = false;
```

© 2025 Zenva Pty Ltd. All rights reserved.

```
          }
        })
      }
    })
  }
}
```

loading-scene.js

Found in folder: **/src/scenes**

This code defines a LoadingScene class that extends Phaser.Scene. The preload() method loads various game assets such as images, spritesheets, and JSON data. The create() method sets up animations for walking and burning, and then starts the GameScene.

```
export class LoadingScene extends Phaser.Scene {
  constructor() {
    super({ key: 'LoadingScene' });
  }

  // load asset files for our game
  preload() {
    // load images
    this.load.image('ground', 'assets/images/ground.png');
    this.load.image('platform', 'assets/images/platform.png');
    this.load.image('block', 'assets/images/block.png');
    this.load.image('goal', 'assets/images/gorilla3.png');
    this.load.image('barrel', 'assets/images/barrel.png');

    // load spritesheets
    this.load.spritesheet('player', 'assets/images/player_spritesheet.png', {
      frameWidth: 28,
      frameHeight: 30,
      margin: 1,
      spacing: 1
    });

    this.load.spritesheet('fire', 'assets/images/fire_spritesheet.png', {
      frameWidth: 20,
      frameHeight: 21,
      margin: 1,
      spacing: 1
    });

    // load json data
    this.load.json('levelData', 'assets/data/levelData.json');
  }

  // executed once, after assets were loaded
  create() {
    // walking animation
    this.anims.create({
      key: 'walking',
```

© 2025 Zenva Pty Ltd. All rights reserved.

```
      frames: this.anims.generateFrameNames('player', {
        frames: [0, 1, 2]
      }),
      frameRate: 12,
      yoyo: true,
      repeat: -1
    });

    // fire animation
    this.anims.create({
      key: 'burning',
      frames: this.anims.generateFrameNames('fire', {
        frames: [0, 1]
      }),
      frameRate: 4,
      repeat: -1
    });

    this.scene.start("GameScene")
  }
}
```

levelData.json

Found in folder: **/assets/data**

This JSON file defines the initial positions and properties of game objects in a 2D platformer game. It includes the player, ground, platforms, goal, and fires, specifying their x and y coordinates, as well as additional properties like the number of tiles for platforms and the key used to identify each object type. The file also contains settings for the spawner, such as the interval, speed range, and lifespan of spawned objects, and defines the dimensions of the game world.

```
{
  "player": {
    "x": 190,
    "y": 538,
    "key": "player",
    "frame": 0
  },
  "ground": {
    "x": 180,
    "y": 604,
    "key": "ground"
  },
  "platforms": [
    {
      "x": 72,
      "y": 450,
      "numOfTiles": 6,
      "key": "block"
    },
    {
      "x": 0,
```

© 2025 Zenva Pty Ltd. All rights reserved.

```
      "y": 330,
      "numOfTiles": 8,
      "key": "block"
    },
    {
      "x": 72,
      "y": 210,
      "numOfTiles": 8,
      "key": "block"
    },
    {
      "x": 0,
      "y": 90,
      "numOfTiles": 7,
      "key": "block"
    }
  ],
  "goal": {
    "x": 30,
    "y": 65,
    "key": "goal"
  },
  "fires": [
    {
      "x": 12.9,
      "y": 545,
      "key": "fire"
    },
    {
      "x": 328,
      "y": 188,
      "key": "fire"
    },
    {
      "x": 68.3,
      "y": 307,
      "key": "fire"
    },
    {
      "x": 87.7,
      "y": 427,
      "key": "fire"
    }
  ],
  "spawner": {
    "interval": 2000,
    "speed": {
      "min": 90,
      "max": 120
    },
    "lifespan": 30000
  },
  "world": {
    "width": 360,
    "height": 700
```


© 2025 Zenva Pty Ltd. All rights reserved.

```
  }
}
```

© 2025 Zenva Pty Ltd. All rights reserved.